Application Security Controls

Application security controls protect against common vulnerabilities and web application attacks. This section creates comprehensive specifications for preventing OWASP Top 10 vulnerabilities, implementing secure coding practices, and establishing security testing procedures.

Exercise 3.1: OWASP Top 10 Protection

Implement comprehensive protection against OWASP Top 10 vulnerabilities:

~~~
Create detailed protection against OWASP Top 10 vulnerabilities:

**A01: Broken Access Control Protection**:
```typescript
interface AccessControlProtection {
  // Authorization Implementation
  authorizationControls: {
    principleOfLeastPrivilege: "Grant minimum necessary permissions";
    defaultDeny: "Deny access by default, explicitly grant permissions";
    resourceOwnership: "Verify user owns resources before access";
    sessionValidation: "Validate session and permissions on every request";
  };

  // Implementation Patterns
  implementationPatterns: {
    middlewareProtection: {
      authentication: "Verify JWT token on protected routes";
      authorization: "Check user permissions for specific resources";
      rateLimiting: "Prevent abuse through rate limiting";
      logging: "Log all access attempts for audit";
    };

    apiProtection: {
      endpointSecurity: "Protect all API endpoints with appropriate controls";
      parameterValidation: "Validate all input parameters";
      responseFiltering: "Filter response data based on user permissions";
      corsConfiguration: "Proper CORS configuration";
    };
  };
}

// A02: Cryptographic Failures Protection
interface CryptographicProtection {
  dataProtection: {
    encryptionAtRest: "AES-256 encryption for sensitive data";
    encryptionInTransit: "TLS 1.3 for all communications";
    keyManagement: "Proper key rotation and storage";
    saltedHashing: "bcrypt with salt for password hashing";
  };

  implementation: {
    sensitiveDataIdentification: "Classify and protect all sensitive data";
    encryptionStandards: "Use industry-standard encryption algorithms";
    keyStorage: "Store keys separately from encrypted data";
    certificateManagement: "Proper SSL/TLS certificate management";
  };
}

// A03: Injection Protection
interface InjectionProtection {
  preventionMeasures: {
    parameterizedQueries: "Use parameterized queries for database access";
    inputValidation: "Validate and sanitize all user inputs";
    outputEncoding: "Encode output based on context";
    principleOfLeastPrivilege: "Use least privileged database accounts";
  };

  specificProtections: {
    sqlInjection: "Parameterized queries and input validation";
    nosqlInjection: "Input validation and query sanitization";
    commandInjection: "Avoid system calls, use safe APIs";
    ldapInjection: "Input validation and parameterized queries";
  };
}
```
~~~

~~~
**A04-A10 Protection Strategies**:
```typescript
interface RemainingOWASPProtection {
  // A04: Insecure Design
  secureDesign: {
    threatModeling: "Systematic threat analysis during design";
    securityRequirements: "Define security requirements early";
    secureArchitecture: "Design with security principles";
    paved_road: "Use secure, well-tested patterns";
  };

  // A05: Security Misconfiguration
  configurationSecurity: {
    hardeningProcedures: "Systematic security hardening";
    defaultCredentials: "Change all default passwords and configurations";
    errorHandling: "Secure error messages without information disclosure";
    securityHeaders: "Implement all required security headers";
  };

  // A06: Vulnerable Components
  componentSecurity: {
    inventoryManagement: "Maintain inventory of all components";
    vulnerabilityScanning: "Regular vulnerability scanning";
    updateProcedures: "Systematic update and patching procedures";
    sourceValidation: "Verify component sources and integrity";
  };

  // A07: Authentication Failures
  authenticationSecurity: {
    strongAuthentication: "Multi-factor authentication implementation";
    sessionManagement: "Secure session handling";
    passwordPolicies: "Strong password requirements";
    bruteForceProtection: "Account lockout and rate limiting";
  };

  // A08: Software Integrity Failures
  integrityProtection: {
    codeSignature: "Digital signatures for code and updates";
    dependencyValidation: "Verify integrity of dependencies";
    cicdSecurity: "Secure CI/CD pipeline implementation";
    updateMechanism: "Secure update and deployment processes";
  };

  // A09: Logging Failures
  loggingSecurity: {
    comprehensiveLogging: "Log all security-relevant events";
    logProtection: "Protect log files from tampering";
    logMonitoring: "Real-time monitoring and alerting";
    logRetention: "Appropriate log retention policies";
  };

  // A10: Server-Side Request Forgery
  ssrfProtection: {
    inputValidation: "Validate and sanitize URLs";
    networkSegmentation: "Isolate application from internal networks";
    allowlisting: "Use allowlists for external requests";
    responseValidation: "Validate responses from external services";
  };
}
```

Research current OWASP Top 10 vulnerabilities and protection strategies using the Web Search MCP server.
~~~

Expected Outcome

- Comprehensive protection against all OWASP Top 10 vulnerabilities
- Secure coding practices integrated throughout the application
- Systematic security controls with proper implementation
- Regular security testing and vulnerability assessment
- Continuous monitoring for new threats and vulnerabilities

Exercise 3.2: Input Validation and Output Encoding

Implement comprehensive input validation and output encoding:

~~~
Create detailed input validation and output encoding specifications:

**Input Validation Framework**:
```typescript
interface InputValidationFramework {
  // Validation Strategies
  validationStrategies: {
    whitelistValidation: "Accept only known good input";
~~~

~~~
    lengthValidation: "Enforce appropriate length limits";
    formatValidation: "Validate input format and structure";
    businessLogicValidation: "Validate against business rules";
  };

  // Implementation Patterns
  validationPatterns: {
    serverSideValidation: {
      mandatory: "All validation must occur on server side";
      clientSideSupport: "Client-side validation for UX only";
      redundantValidation: "Validate at multiple layers";
      failSecure: "Fail securely when validation fails";
    };

    inputSanitization: {
      htmlSanitization: "Remove or encode HTML tags";
      sqlSanitization: "Escape SQL special characters";
      scriptSanitization: "Remove or neutralize script content";
      pathSanitization: "Validate and sanitize file paths";
    };
  };

  // Validation Rules
  validationRules: {
    emailValidation: {
      format: "RFC 5322 compliant email format";
      length: "Maximum 254 characters";
      domainValidation: "Validate domain existence";
      disposableCheck: "Check against disposable email providers";
    };

    passwordValidation: {
      length: "Minimum 12 characters";
      complexity: "Mixed case, numbers, special characters";
      commonPasswords: "Check against common password lists";
      personalInfo: "Prevent use of personal information";
    };

    messageValidation: {
      length: "1-4000 characters";
      contentFiltering: "Filter inappropriate content";
      encodingValidation: "Validate character encoding";
      malwareScanning: "Scan for malicious content";
    };
  };
}
```

**Output Encoding Specifications**:
```typescript
interface OutputEncodingSpecifications {
  // Context-Specific Encoding
  contextualEncoding: {
    htmlContext: {
      encoding: "HTML entity encoding for content";
      attributes: "Attribute encoding for HTML attributes";
      javascript: "JavaScript encoding for script contexts";
      css: "CSS encoding for style contexts";
    };

    jsonContext: {
      encoding: "JSON encoding for API responses";
      escaping: "Proper escaping of special characters";
      validation: "Validate JSON structure";
    };

    urlContext: {
      encoding: "URL encoding for query parameters";
      pathEncoding: "Path encoding for URL paths";
      validation: "Validate URL structure and format";
    };
  };

  // Encoding Implementation
  encodingImplementation: {
    automaticEncoding: "Automatic encoding based on output context";
    templateSecurity: "Secure templating with auto-escaping";
    apiSecurity: "Secure API response encoding";
    errorHandling: "Secure error message encoding";
  };
}
```

Research current input validation and output encoding best practices.
~~~

Expected Outcome

- Comprehensive input validation framework with multiple validation layers
- Context-aware output encoding preventing injection attacks
- Automated validation and encoding integrated into the application
- Protection against malicious input and content filtering
- Secure error handling without information disclosure

Exercise 3.3: Security Testing and Code Review

Implement comprehensive security testing and code review processes:

~~~
Create detailed security testing and code review specifications:

**Security Testing Framework**:
```typescript
interface SecurityTestingFramework {
  // Static Application Security Testing (SAST)
  sastTesting: {
    tools: ["SonarQube", "ESLint security rules", "Semgrep"];
    coverage: "Scan all source code for security vulnerabilities";
    integration: "Integrate into CI/CD pipeline";
    reporting: "Detailed vulnerability reports with remediation guidance";
  };

  // Dynamic Application Security Testing (DAST)
  dastTesting: {
    tools: ["OWASP ZAP", "Burp Suite", "Nessus"];
    scope: "Test running application for vulnerabilities";
    automation: "Automated scanning in staging environment";
    authentication: "Authenticated and unauthenticated testing";
  };

  // Interactive Application Security Testing (IAST)
  iastTesting: {
    implementation: "Runtime security testing during development";
    coverage: "Real-time vulnerability detection";
    integration: "Integrate with development workflow";
  };

  // Penetration Testing
  penetrationTesting: {
    frequency: "Quarterly external penetration testing";
    scope: "Complete application and infrastructure testing";
    methodology: "OWASP Testing Guide methodology";
    reporting: "Detailed findings with risk assessment";
  };
}
```

**Security Code Review Process**:
```typescript
interface SecurityCodeReview {
  // Review Process
  reviewProcess: {
    mandatoryReview: "All code changes require security review";
    reviewers: "Security-trained developers and security team";
    checklist: "Comprehensive security review checklist";
    tooling: "Automated security analysis tools";
  };

  // Review Checklist
  securityChecklist: {
    authentication: "Proper authentication implementation";
    authorization: "Correct authorization checks";
    inputValidation: "Comprehensive input validation";
    outputEncoding: "Proper output encoding";
    errorHandling: "Secure error handling";
    cryptography: "Correct cryptographic implementation";
    logging: "Appropriate security logging";
    configuration: "Secure configuration management";
  };

  // Automated Security Checks
  automatedChecks: {
    linting: "Security-focused linting rules";
    dependencyScanning: "Vulnerability scanning of dependencies";
    secretScanning: "Scan for hardcoded secrets";
    licenseCompliance: "Check for license compliance issues";
  };
}
```

Research current security testing methodologies and tools.
~~~

Expected Outcome

- Comprehensive security testing framework with multiple testing types
- Automated security testing integrated into CI/CD pipeline
- Systematic security code review process with checklists
- Regular penetration testing and vulnerability assessments
- Continuous security monitoring and improvement processes

Exercise 3.4: Security Implementation

Implement all application security controls based on specifications:

~~~
Based on all security specifications, implement comprehensive application security:

**Step 1: OWASP Protection Implementation**
Implement protection against all OWASP Top 10 vulnerabilities:

```typescript
// src/middleware/security.ts
export const securityMiddleware = {
  // A01: Access Control
  accessControl: (req: Request, res: Response, next: NextFunction) => {
    // Implement authorization checks as specified
    const token = extractToken(req);
    const user = validateToken(token);
    const hasPermission = checkPermissions(user, req.path, req.method);

    if (!hasPermission) {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }

    req.user = user;
    next();
  },

  // A03: Injection Protection
  inputValidation: (req: Request, res: Response, next: NextFunction) => {
    // Implement input validation as specified
    const validationResult = validateInput(req.body, req.params, req.query);

    if (!validationResult.isValid) {
      return res.status(400).json({
        error: 'Validation failed',
        details: validationResult.errors
      });
    }

    next();
  },

  // Security headers
  securityHeaders: helmet({
    contentSecurityPolicy: {
      directives: {
        defaultSrc: ["'self'"],
        scriptSrc: ["'self'"],
        styleSrc: ["'self'", "'unsafe-inline'"],
        // Implement all CSP directives as specified
      },
    },
    hsts: {
      maxAge: 31536000,
      includeSubDomains: true,
      preload: true
    }
  })
};
```

**Step 2: Input Validation Implementation**
Implement comprehensive input validation:

```typescript
// src/validation/schemas.ts
export const validationSchemas = {
  userRegistration: {
    email: {
      type: 'string',
      format: 'email',
      maxLength: 254,
      // Implement all email validation rules as specified
    },
    password: {
      type: 'string',
~~~

~~~
      minLength: 12,
      pattern: '^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@$!%*?&])[A-Za-z\\d@$!%*?&]',
      // Implement all password validation rules as specified
    }
  },

  chatMessage: {
    content: {
      type: 'string',
      minLength: 1,
      maxLength: 4000,
      // Implement content filtering and validation as specified
    }
  }
};

// src/validation/validator.ts
export class InputValidator {
  static validate(data: any, schema: any): ValidationResult {
    // Implement validation logic exactly as specified
    // - Schema validation
    // - Business rule validation
    // - Security validation
    // - Sanitization
  }

  static sanitize(input: string, context: 'html' | 'sql' | 'javascript'): string {
    // Implement context-aware sanitization as specified
  }
}
```

**Step 3: Security Testing Implementation**
Implement automated security testing:

```typescript
// tests/security/security.test.ts
describe('Security Tests', () => {
  describe('OWASP Top 10 Protection', () => {
    it('should prevent SQL injection attacks', async () => {
      // Implement security tests as specified
      const maliciousInput = "'; DROP TABLE users; --";
      const response = await request(app)
        .post('/api/v1/chat/message')
        .send({ content: maliciousInput });

      expect(response.status).toBe(400);
      expect(response.body.error).toContain('Validation failed');
    });

    it('should prevent XSS attacks', async () => {
      // Implement XSS protection tests as specified
    });

    // Implement all OWASP Top 10 tests as specified
  });

  describe('Authentication Security', () => {
    it('should enforce strong password requirements', async () => {
      // Implement authentication security tests as specified
    });

    it('should prevent brute force attacks', async () => {
      // Implement brute force protection tests as specified
    });
  });
});

// Security scanning integration
// package.json scripts
{
  "scripts": {
    "security:scan": "npm audit && snyk test",
    "security:sast": "sonar-scanner",
    "security:dast": "zap-baseline.py -t http://localhost:3000",
    "security:all": "npm run security:scan && npm run security:sast"
  }
}
```

**Implementation Checklist**:
- [ ] All OWASP Top 10 protections implemented and tested
- [ ] Comprehensive input validation and output encoding
- [ ] Security middleware protecting all endpoints
- [ ] Automated security testing in CI/CD pipeline
- [ ] Security headers and HTTPS enforcement
- [ ] Error handling without information disclosure
- [ ] Logging and monitoring for security events
- [ ] Regular security scanning and updates
~~~

```
- [ ] Penetration testing procedures established
- [ ] Security incident response procedures

Use Agent Hooks to automatically run security tests when code changes and validate security configurations.
```

Expected Outcome

- Complete application security implementation matching all specifications
- Protection against OWASP Top 10 and other common vulnerabilities
- Automated security testing integrated into development workflow
- Comprehensive input validation and output encoding
- Security monitoring and incident response capabilities

Key Deliverables

After completing the Application Security exercises, you should have:

1. **OWASP Top 10 Protection**: Comprehensive protection against all major web vulnerabilities
2. **Input Validation Framework**: Robust validation and sanitization for all user inputs
3. **Security Testing Suite**: Automated security testing with SAST, DAST, and penetration testing
4. **Security Implementation**: Complete security controls integrated throughout the application
5. **Security Monitoring**: Continuous security monitoring and incident response capabilities

Next Steps

With comprehensive application security controls established, you're ready to implement compliance and governance frameworks that ensure regulatory compliance, audit trails, and systematic security management.

Previous Next

Ascertain Kiro Workshop

Agentic Coding with Kiro
Introduction to Kiro
Setup
Vibe Coding
Spec Coding
Foundation
Infrastructure
Frontend
Backend
Security
Authentication and

Event ends in 2 days 7 hours 58 minutes.

© 2008 - 2026, Amazon Web Services, Inc. or its affiliates. All rights reserved. Privacy policy Terms of use Cookie preferences